Hybrid CPU-GPU Based Real-Time Crowd Simulation

Course: Parallel and Distributed Computing (UCS645)

Department: DCSE, Thapar Institute of Engineering and Technology, Patiala

Problem Statement: The Challenge of Simulating Large Crowds

Crowd simulation and swarm simulation are critical computational tools used across diverse domains including pedestrian movement analysis, swarm robotics, and autonomous systems. These systems must continuously update thousands of individual agents whilst ensuring each agent behaves realistically according to predefined behavioural rules.

Behavioural Rules

Each agent follows specific rules: avoiding collisions, moving towards targets, and maintaining group cohesion

Computational Burden

As agent count increases, computational complexity grows exponentially, creating massive processing demands

Real-Time Barrier

Traditional CPU-based systems struggle to achieve real-time performance with large agent populations

Project Objective: Accelerating Crowd Simulation

GPU Acceleration

Leverage GPU parallel processing to dramatically accelerate computation

Hybrid Architecture

Develop a system that intelligently combines CPU and GPU resources

Scalability

Ensure efficient operation with thousands of simultaneous agents

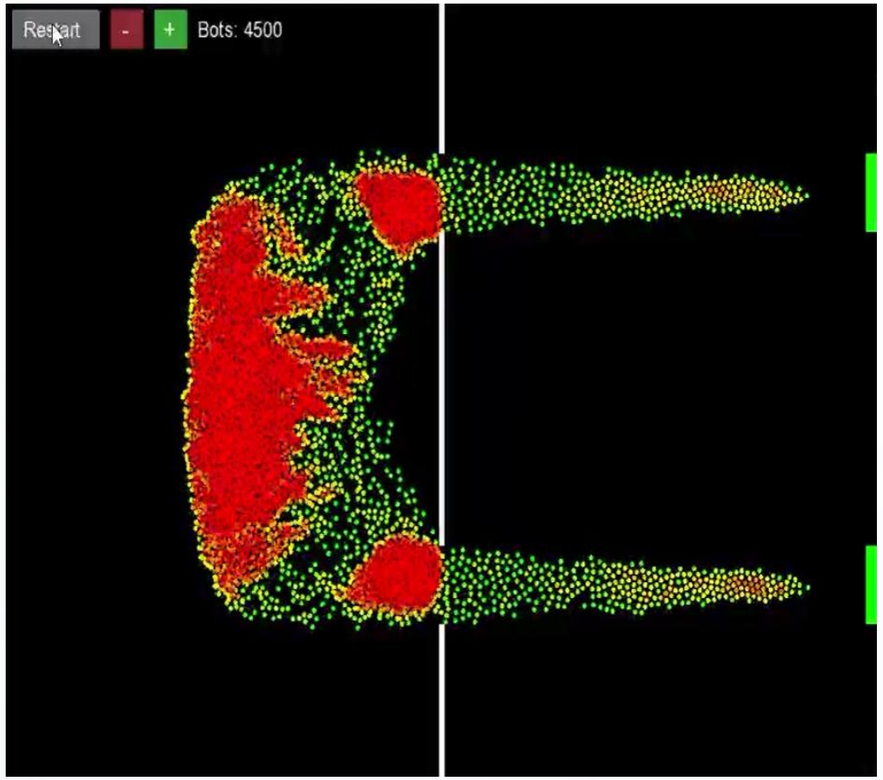
Performance Analysis

Compare against traditional CPU-based approaches quantitatively

This project demonstrates distributed computing principles by implementing Hybrid CPU–GPU Based Real-Time Crowd Simulation, addressing real-world computational challenges through algorithmic parallelisation.

The Boids Model

The simulation is built upon the Boids model, a well-established approach for simulating flocking and group behaviour in nature. This model provides a foundation for realistic crowd dynamics through simple, localised rules that produce emergent collective behaviour.



Hybrid Processing Strategy

Our approach employs a strategic division of labour between CPU and GPU components. The GPU handles the computationally intensive agent updates and neighbour interactions, whilst the CPU manages simulation control, parameter adjustments, and visualisation coordination. This workload distribution maximises resource utilisation and system efficiency.

01

GPU: Parallel Computation

Process thousands of agents simultaneously

02

CPU: System Control

Coordinate simulation flow and parameters

03

Integration: Real-Time

Combine outputs for seamless simulation

GPU-Based Parallelisation: The Computational Engine

The GPU serves as the computational powerhouse, executing the most demanding aspects of the simulation through massive parallel processing capabilities. Modern GPUs contain thousands of cores that can process multiple agents simultaneously, transforming what would be sequential operations into parallelised computations.

Position Updates

Simultaneously update positions and velocities of all agents at each timestep

Neighbour Interactions

Calculate interactions between neighbouring agents efficiently

Behaviour Rules

Apply separation, alignment, and cohesion rules across all agents

Collision Avoidance

Process collision detection and avoidance in parallel

Scalable Computation

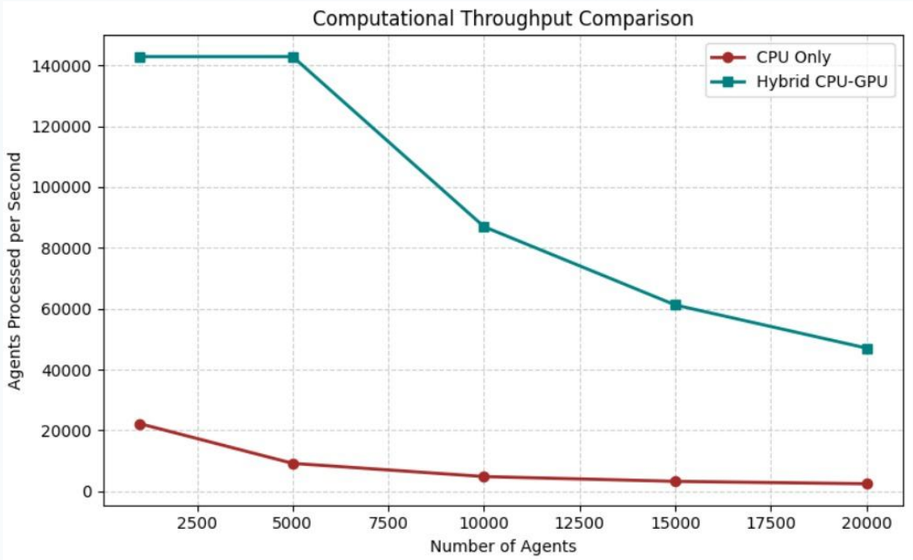
Scale numerical computations to thousands of concurrent agents

Reduced Execution

Drastically reduce simulation execution time through parallelisation

CPU-Based Control Logic: The Orchestrator

Whilst the GPU performs heavy computational lifting, the CPU acts as the intelligent controller, managing the overall simulation framework and coordinating between components. This division of responsibilities mirrors real-world distributed computing architectures.

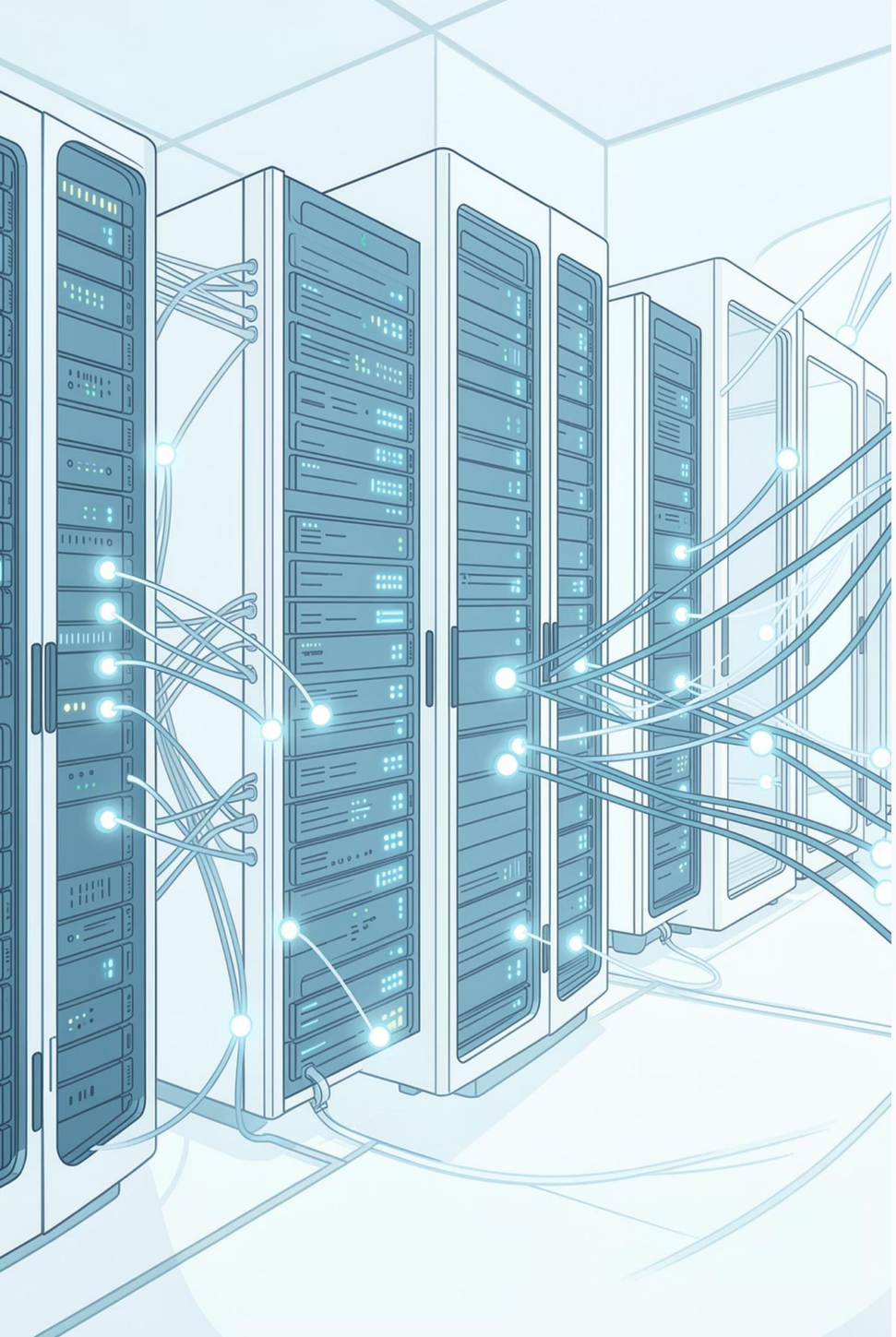


Simulation Management
Controls overall simulation flow, updates parameters, and handles user input

Environment Handling
Manages boundaries, environmental conditions, and obstacle configurations

Step Coordination
Orchestrates the sequence of simulation steps and synchronises GPU computations

Visualisation Interface
Handles rendering, display updates, and user interaction with the simulation



Parallel and Distributed Computing Relevance

This project exemplifies fundamental principles of parallel and distributed computing through practical implementation. It demonstrates how traditional algorithms can be transformed for modern hardware architectures, providing insights applicable to numerous computational domains.

Algorithm Parallelisation

Transforms sequential Boids model into parallelised computation suitable for GPU architecture

Heterogeneous Computing

Illustrates efficient collaboration between different processor types with complementary strengths

Workload Distribution

Strategically allocates tasks across processing units based on computational requirements

Real-World Application

Addresses practical computational challenges in crowd simulation and autonomous systems

Performance Evaluation Metrics

The system undergoes rigorous testing using standardised performance metrics that quantify real-time simulation capabilities. These metrics provide objective measures of improvement over CPU-only approaches.

1

Frame Time

Definition: Time required to process one simulation timestep

Target: Lower values indicate better performance

Measurement: Milliseconds per frame

2

Frames Per Second (FPS)

Definition: Number of simulation frames processed per second

Target: Higher FPS indicates smoother, more responsive simulation

Threshold: 30+ FPS for real-time visualisation

3

Scalability Analysis

Definition: System's ability to handle increasing agent counts efficiently

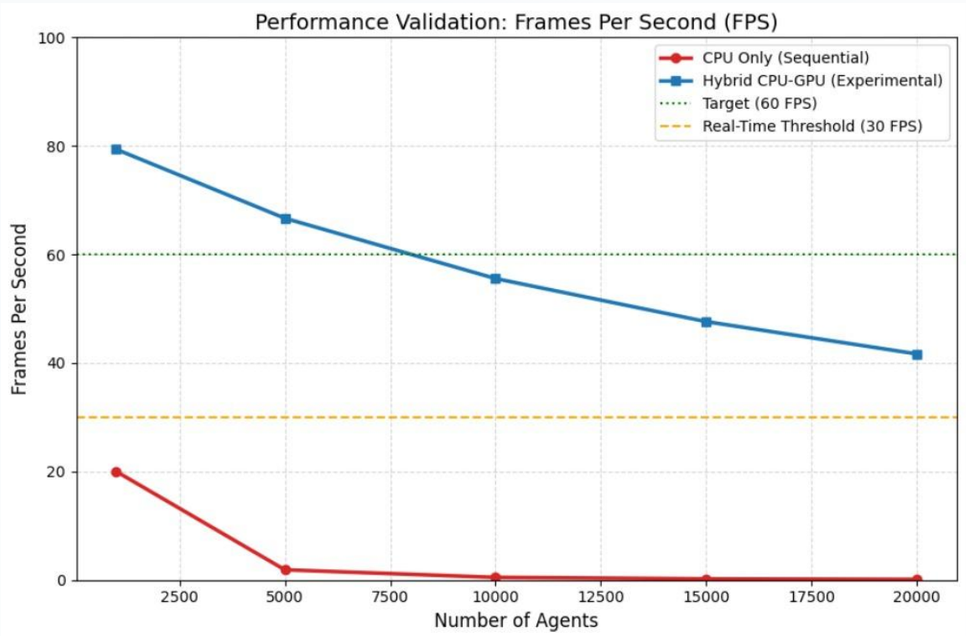
Testing Range: 500 to 10,000 agents

Goal: Maintain acceptable frame rates as complexity increases

Uniqueness and Expected Outcomes

What Sets This Project Apart

Unlike approaches that rely on AI models or deep learning techniques, this project focuses on algorithmic improvements at the computational level. The hybrid CPU–GPU architecture provides a general-purpose solution applicable to various crowd simulation scenarios without requiring specialised training data or complex neural networks.



Real-Time Large-Scale Simulation

Achieve real-time performance with thousands of simultaneous agents, enabling practical applications in pedestrian analysis and autonomous systems

Measurable Performance Gains

Demonstrate significant speedup ratios compared to traditional CPU-only implementations, with quantifiable reductions in frame processing time

Enhanced Scalability

Showcase superior scalability characteristics, maintaining acceptable frame rates even as agent populations increase substantially beyond CPU-only capabilities

Conclusion: Bridging Algorithm and Architecture

This project demonstrates how intelligent workload distribution between CPU and GPU processors can transform computationally intensive crowd simulation from impractical to real-time. By applying parallel computing principles to the established Boids model, we achieve:

Substantial Performance Gains

Through GPU parallelisation of agent updates and neighbour calculations

Scalable Architecture

Capable of handling thousands of agents efficiently

Practical Implementation

Without relying on AI or deep learning dependencies

The Hybrid CPU–GPU Based Real-Time Crowd Simulation approach exemplifies distributed computing principles whilst addressing real-world computational challenges. This methodology provides a foundation applicable to numerous domains requiring large-scale agent-based simulation, from urban planning to autonomous vehicle coordination.